```
manifest.json
```

It is the most important file in a Chrome Extension.

Think:

```
manifest.json 📄
```

↓

```
Extension configuration file
```

↓

```
Tells Chrome:

Who am I?
What permissions do I need?
Which files should run?
Which websites can I access?
```

🔹 **manifest_version**

```
"manifest_version": 3
```

```
manifest_version
```

→ predefined Chrome Extension property

```
3
```

→ Manifest V3 format

Think:

```
Chrome:
Which extension format are you using?

Extension:
Version 3
```

🔹 **name**

```
"name": "Auto Spam Detector"
```

```
name
```

→ predefined property

Value:

```
Auto Spam Detector
```

This is what Chrome displays.

Example:

```
Extensions
↓
Auto Spam Detector
```

🔹 **version** 

```
"version": "1.0"
```

```
version
```

→ predefined property

Used by Chrome to track extension versions.

Example:

```
1.0
1.1
2.0
```

🔹 **permissions**

```
"permissions": ["activeTab"]
```


```
permissions
```

→ predefined property

Chrome asks:

```
What powers do you need?
```

Here:

```
activeTab
```

means:

```
Allow extension to work
with currently active tab
```

Example:

If Gmail tab is open:

```
activeTab
```

allows interaction with that tab.

🔹 **host_permissions**

```
"host_permissions": [
    "http://3.110.181.26:8080/*"
]
```

// "http://localhost:8080/*"

Very important.

Chrome extensions cannot call random servers.

Without permission:

```
fetch("http://3.110.181.26:8080")
```

↓

Blocked ❌

So you tell Chrome:

```
I want permission
to communicate with:

http://3.110.181.26:8080/*
```

Meaning:

```
http://3.110.181.26:8080/email/analyze
```

Allowed ✅

```
http://3.110.181.26:8080/anything
```

Allowed ✅

What does:

```
*
```

mean?

Wildcard.

Think:

```
Everything after /
```

Example:

```
http://3.110.181.26:8080/test

http://3.110.181.26:8080/email/analyze

http://3.110.181.26:8080/abc
```

All allowed.

🔹 **content_scripts**

```
"content_scripts": [
```

```
content_scripts
```

→ predefined property

Think:


```
Which JavaScript files
should Chrome inject
into webpages?
```

Your extension answers:

```
content.js
```

🔹 **matches**

```
"matches": ["https://mail.google.com/*"]
```

Very important.

Means:

```
Only run on Gmail
```

Example:

Gmail:

```
https://mail.google.com/
```

Matches ✅

YouTube:

```
https://youtube.com
```

Matches ❌

Facebook:

```
https://facebook.com
```

Matches ❌

So your extension runs only on Gmail.

🔹 **js**

```
"js": ["content.js"]
```

Means:

```
Inject content.js
into matching pages
```

When Gmail opens:

Chrome automatically loads:

```
content.js
```

Your code starts running.

So:

```
console.log("GLOBAL Spam Detector Running");
```

runs because:

```
manifest.json
↓
content_scripts
↓
content.js
↓
execute
```

🔹 **run_at**

```
"run_at": "document_idle"
```

```
run_at
```

→ predefined property

Controls:

```
When should content.js run?
```

Value:

```
document_idle
```

means:

```
Wait until page
has mostly finished loading
```

Then execute:

```
content.js
```

Why?

Because your code uses:

```
document.querySelector(".a3s.aiL")
```

to find Gmail email content.

If script runs too early:

```
Email HTML
not loaded yet
```

Then:

```
querySelector()
```

finds nothing.

So:

```
document_idle
```

means:

```
Run after webpage is ready
```

🔹 COMPLETE FLOW

```
Chrome starts
↓
Reads manifest.json
↓
Sees matches:
https://mail.google.com/*
↓
User opens Gmail
↓
Chrome injects content.js
↓
document_idle reached
↓
content.js runs
↓
Extract email text
↓
fetch()
↓
Spring Boot
↓
Flask ML
↓
Prediction
↓
Create spam-box
↓
document.body.appendChild(box)
↓
Spam: SPAM
```

This manifest file is basically the "instruction sheet" that tells Chrome how your extension should behave. ✅